Recipe 3: Configuring Custom Task Automation Scripts

While Laravel Scheduler is very well handling the standard tasks very efficiently, there will be instances where you need more specialized automation scripts. Some tasks may require the use of custom shell commands that could perform specific actions not natively covered by Laravel's Artisan commands such as complex file manipulations, running system-level maintenance operations, or executing third-party utility commands that complement your Laravel operations. So here, we need a flexible system, one that could run custom shell scripts and integrate their execution within the Laravel command infrastructure. We are now going to learn how to design and execute custom shell scripts and integrate them with Laravel commands. It will provide you with the capability to automate unique and complex tasks with ease while keeping all automated processes organized and consistent.

Creating Custom Shell Script

First, let's create a custom shell script that performs a unique task. For our demonstration, imagine you need a script that archives old images from your media directory into a compressed file for backup or cleanup purposes.